Le langage Python

Structures de contrôle

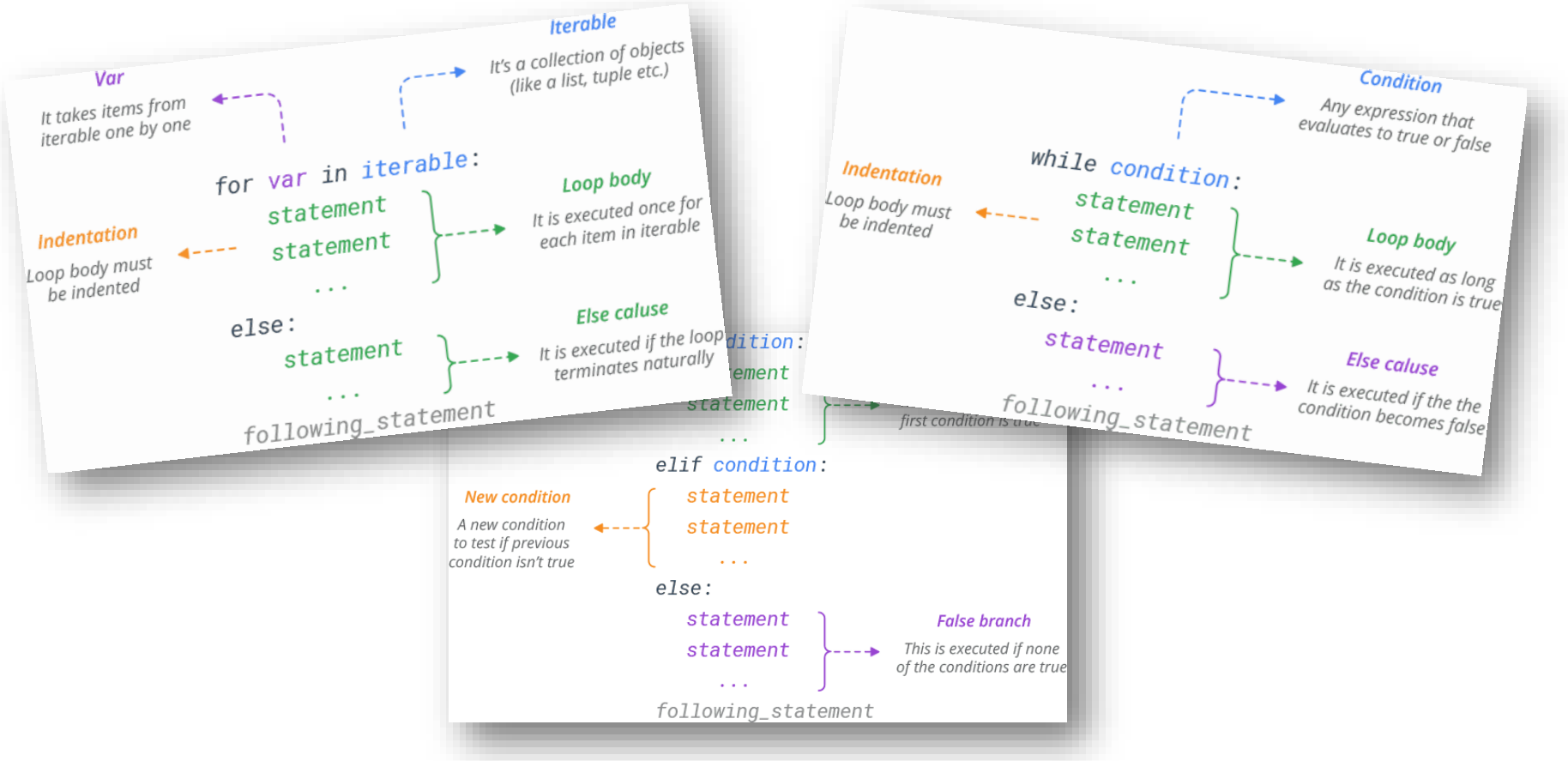
Dominique FRANCISCI

(2020-2026)



Objectif de ce support

Etre capable d'utiliser les structures de contrôle Python



Sommaire



- **Séquence d'instructions**
- Opérateur de sélection
- La structure if-elif-else
- Opérateurs de répétition
- Table de valeurs

Séquence d'instructions

- Les instructions d'un script s'exécutent les unes après les autres
- L'ordre d'écriture détermine l'ordre d'exécution
- Modifier l'ordre relatif de deux instructions peut complètement changer la signification d'un programme

Exemple : script 1



```
x,y = 1,2
x = y
y = x
print(x,y)


>>>
  2 2
```

Exemple : script 2



```
x,y = 1,2
y = x
x = y
print(x,y)


>>>
  1 1
```

Séquence d'instructions

- Il est nécessaire de pouvoir modifier l'ordre d'exécution (le flux d'exécution) pour écrire des programmes « pertinents »

- Il existe deux grandes catégories de contrôleurs de flux :
  - Les opérateurs de sélection (if-elif-else en Python)
  - Les opérateurs de répétition (while-for en Python)

Où en sommes-nous ?

- Séquences d'instructions
- **Opérateur de sélection**
- La structure if-elif-else
- Opérateurs de répétition
- Table de valeurs



Opérateur de sélection : l'instruction if

Syntaxe :

- if condition :

  instruction 1

  instruction 2

  …

  instruction n

- « condition » est une donnée de type booléen
- Si condition est vraie,
  alors les instructions 1 à n sont exécutées,
  sinon elles sont ignorées

Exemple : script 1



```
x = 20
if x > 15 :
    x = x-5
    x = x*3
print(x)


>>>
  45
```

Exemple : script 2



```
x = 3
if x > 15 :
    x = x-5
    x = x*3
print(x)


>>>
  3
```

Où en sommes-nous ?

- Séquence d'instructions
- Opérateur de sélection
- **La structure if-elif-else**
- Opérateur de répétition
- Table de valeurs



Opérateur de sélection : l'instruction if-elif-else

Syntaxe :

- **if** condition 1 :

  sequence-d-instructions 1

  elif condition 2 :

  sequence-d-instructions 2

  elif condition n :

  sequence-d-instructions n

  else :

  sequence-d-instructions m

- Une condition est testée seulement si toutes les précédentes sont fausses

- Une seule des séquences d'instructions sera exécutée

Opérateur de sélection : l'instruction if-elif-else

Exemple : script 1

```
x = 10

if x >= 10:
  print('>=10')
elif x>0 and x<10:
  print('0< <10')
else:
  print('<0')

>>>
  >= 10
```

Exemple : script 2

```
x = 5

if x >= 10:
  print('>=10')
elif x>0 and x<10:
  print('0< <10')
else:
  print('<0')

>>>
  0< <10
```

Exemple : script 3

```
x = -2

if x >= 10:
  print('>=10')
elif x>0 and x<10:
  print('0< <10')
else:
  print('<0')

>>>
  <0
```

Opérateur de sélection : l'instruction if-elif-else

Instructions composées :

Une instruction composée est constituée :

- D'une ligne d'entête terminée par le symbole « : »
- D'un bloc d'instructions indenté (décalé) par rapport à la ligne d'entête

Toutes les instructions de même niveau d'indentation font partie du même bloc

Opérateur de sélection : l'instruction if-elif-else

Exemple



```
x = float(input()) # Bloc 1
y = float(input()) # Bloc 1

if x >= y :                 # Bloc 1
        z = x - y           # Bloc 2
        if z == 0 :         # Bloc 2
            print(z)        # Bloc 3
else :                      # Bloc 1
        z = y - x           # Bloc 4
        print(z)            # Bloc 4
```

Opérateur de sélection : l'instruction if-elif-else

Le niveau d'indentation d'une instruction

peut changer le sens d'un programme

Exemple : script 1



```
x,y = 1,2

if x > y :
    x = x + y
    y = y + 1
print(x,y)


>>>
  1 2
```

Exemple : script 2



```
x,y = 1,2

if x > y :
    x = x + y
y = y + 1
print(x,y)


>>>
  1 3
```

Opérateur de sélection : l'instruction if-elif-else

Exercices



1. Ecrire un script qui affiche le plus petit de deux entiers saisis au clavier.

2. Ecrire un script qui permet de résoudre l' équation :

ax+b = 0, avec x réel,

où a et b sont deux réels

saisis au clavier



Opérateur de sélection : l'instruction if-elif-else

Correction exercices



```
# Script qui affiche le plus petit de deux entiers saisis au clavier.
a = int(input('Saisissez un premier entier : '))
b = int(input('Saisissez un second entier : '))

print('Le plus petit des deux entiers est : ', end='')

if a <= b :
  print(a)
else :
  print(b)
```



Opérateur de sélection : l'instruction if-elif-else

Correction exercices

```
# Script qui permet de résoudre l' équation : ax+b = 0, avec x
réel, où a
# et b sont deux réels saisis au clavier
a = int(input('Saisissez a : '))
b = int(input('Saisissez b : '))

if b != 0 :
  if a != 0 :
    print('Une solution unique (-b/a) :', -b/a)
  else :
    print('Pas de solution.')
else :
  if a != 0 :
    print('Solution : 0')
  else :
    print('Infinité de solutions.')
```



Où en sommes-nous ?

- Séquence d'instructions
- Opérateur de sélection
- La structure if-elif-else
- **Opérateurs de répétition**
- Table de valeurs



Opérateur de répétition : l'instruction while

Egalement dénommée « boucle while »

Syntaxe :

- **while** condition :

  instruction 1
  instruction 2
  …
  instruction n

  } Corps de la boucle

- « condition » est une donnée de type booléen
- Si condition est vraie, les instructions 1 à n sont exécutées
- Une fois l'instruction n terminée, si condition est toujours vraie, les instructions 1 à n sont à nouveau exécutées, etc…

Opérateur de répétition : l'instruction while

- « condition » s'appelle le critère de continuation de la boucle
- Les variables composant « condition » doivent toutes avoir une valeur (valeur d'initialisation) lors de la première évaluation de la condition de continuation
- Il faut impérativement s'assurer que la boucle se termine
- Une boucle ne se terminant pas s'appelle une « boucle infinie »

Opérateur de répétition : l'instruction while

Exemple : script 1



```
x = 5
while x > 0 :
    x = x - 1
    print(x)


>>>
  4
  3
  2
  1
  0
```

Exemple : script 2



```
x = 5
while x < 3 :
    x = x + 1
    print(x)


>>>
```

Opérateur de répétition : l'instruction for

Egalement dénommée « boucle for »

Syntaxe :

- **for** x **in** ensemble :

  instruction 1
  instruction 2
  …
  instruction n

  } Corps de la boucle

- x est un élément de ensemble
- Les instructions 1 à n sont exécutées pour chaque élément x de l'ensemble ensemble
- « ensemble » peut être une chaîne de caractères, une structure de données (cf. support « Structures de données ») ou tout autre type incluant des itérateurs

Opérateur de répétition : l'instruction for

Exemple



```
string = 'Hello world ! '

for c in string :
    print(c, end='')

>>>
  Hello world !
```

Où en sommes-nous ?

- Séquence d'instructions
- Opérateur de sélection
- La structure if-elif-else
- Opérateurs de répétition
- **Table de valeurs**



Table de valeurs

- Une table des valeurs permet de suivre les modifications des variables ou expressions au cours de l'exécution d'un script

Exemple



```
x = 3
while x > 0 :
  x = x - 1
  print(x)

>>>
  2
  1
  0
```

Table des valeurs :

x	x > 0	Ecran
3	True	
2		2
	True	
1		1
	True	
0		0
	False	

Exemple d'itérations

Exercice (1/3)



Ecrire un script qui affiche tous les entiers entre 1 et 100.



Exemple d'itérations

Correction exercice 1/3

```
# Script qui affiche tous les entiers entre 1 et 100.
n = 1

while n <= 100 :
  print(n)
  n = n + 1

>>>
  1
  2
  3
  ...
  99
  100
```



Exemple d'itérations

Exercice (2/3)



Ecrire un script qui affiche la somme des entiers entre 1 et 100.



Exemple d'itérations

Correction exercice 2/3

```
# Script qui affiche la somme des entiers entre 1 et 100.
n = 1
total = 0

while n <= 100 :
  total = total + n
  n = n + 1

print(total)

>>>
  5050
```



Exemple d'itérations

Exercice (3/3)



Ecrire un script qui calcule la somme des entiers saisis par l'utilisateur, la saisie s'arrête quand l'utilisateur saisit 0.



Exemple d'itérations

Correction exercice 3/3

```
# Calcule la somme des entiers saisis par l'utilisateur,
# la saisie s'arrête quand l'utilisateur saisit 0.
cont  = True
total = 0

while cont :
  a = int(input('Saisissez un entier : '))
  total = total + a
  cont = (a != 0)

print(total)
```



Boucles imbriquées

Exercice



Répéter un bloc d'instructions contenant lui-même une boucle while :

1. *Ecrire un script qui affiche la table de multiplication d'un entier n saisi au clavier.*
2. *Modifier le script pour qu'il affiche toutes les tables des entiers entre 1 et 10.*



Boucles imbriquées

Correction exercice (version 1)

```
# Table de n.
n = int(input())
i = 1

while i <= 10 :
  print(n, '*', i, '=', n*i)
  i = i + 1
```

Correction exercice (version 2)

```
# Table des nombres entre 1 et 10.
i = 1
n = 1

while n <= 10 :
  i = 1
  while i <= 10 :
    print(n, '*', i, '=', n*i)
    i = i + 1
  n = n + 1
```



Liste des nombres premiers

Exercice



On dit qu'un entier n >= 2 est premier s'il n'est divisible que par 1 et par lui-même.

On veut afficher tous les nombres premiers inférieurs à un entier n saisi au clavier :

- *Ecrire un script permettant de savoir si un entier n est premier.*
- *Répéter le bloc de code précédent pour tous les entiers entre 2 et n.*



Liste des nombres premiers

Correction exercice

```
# n est-il premier ?
n = int(input('Entrez un entier : '))

div = 2
est_premier = True

while div < n and est_premier :
  est_premier = (n%div != 0)
  div = div + 1

if est_premier == True :
  print(n, 'est premier')
else :
  print(n, 'n'est pas premier')
```

Correction exercice

```
# Liste des nombres premiers.
n_max = int(
   input('Entrez un entier max : '))
n = 2

print('Nombres premiers <=', n_max)
while n <= n_max :
  div = 2
  est_premier = True
  while div < n and est_premier :
    est_premier = (n%div != 0)
    div = div + 1
  if est_premier == True :
    print(n, end=' ')
  n = n + 1
```

